```
import numpy as np
import os
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import IsolationForest
from sklearn.metrics import confusion_matrix, classification_report
import time

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report,
f1_score,roc_auc_score
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.callbacks import EarlyStopping
import random
from sklearn.model_selection import train_test_split
from tensorflow.keras import initializers
from sklearn.metrics import precision_score, recall_score, f1_score

seed = 42

os.environ['PYTHONHASHSEED'] = str(seed)
os.environ['TF_DETERMINISTIC_OPS'] = '1'

random.seed(seed)
np.random.seed(seed)
tf.random.set_seed(seed)
```

Build different Normalities

```
def load_timeseries_from_folders(normality, folder_list):
    all_dfs = []
    folders_to_read = folder_list[:normality]

    for folder in folders_to_read:
        # Full folder path
        folder_path = os.path.join("..", "TSB-UAD-Public", folder)

        # List all files in the folder
        files = [f for f in os.listdir(folder_path) if
os.path.isfile(os.path.join(folder_path, f))]

        if not files:
            print(f"Folder {folder} is empty or contains no files.")
            continue

        # Use the first file found
        filename = files[0]
```

```
        file_path = os.path.join(folder_path, filename)

        try:
            df = pd.read_csv(file_path, header=None).dropna()
            df.columns = ['measurement', 'anomaly']
            print(f"Read file {filename} from folder {folder}
({df.shape[0]} rows).")
            all_dfs.append(df)
        except Exception as e:
            print(f"Error reading file {file_path}: {e}")

    if all_dfs:
        full_df = pd.concat(all_dfs, ignore_index=True)
        print("\nData merge completed. Combine df
has",full_df.shape[0]," number of rows")
        return full_df
    else:
        print("No valid data found.")
        return pd.DataFrame()

# Example usage

folders = ["YAHOO","SensorScope","GHL","Genesis"]
normality = 2

result = load_timeseries_from_folders(normality, folders)
result
```

```
Read file YahooA3Benchmark-TS100_data.out from folder YAHOO (1680
rows).
Read file stb-10.test.out from folder SensorScope (29518 rows).

Data merge completed. Combine df has 31198  number of rows
```

	measurement	anomaly
0	6.470061	0
1	80.972293	0
2	159.014680	0
3	214.721803	0
4	201.695834	0
...	...	...
31193	-1.780000	0
31194	-1.460000	0
31195	-1.640000	0
31196	-1.840000	0
31197	-1.340000	0

```
[31198 rows x 2 columns]
```

AE Offline

```
#parameters
window_size = 20
rolling_window = 10
max_epochs = 100

data_raw = result['measurement'].values
labels = result['anomaly'].values
n_samples = len(data_raw)

start_time = time.time()


#Features
rolling_mean = pd.Series(data_raw).rolling(window=rolling_window,
min_periods=1).mean().values
rolling_std = pd.Series(data_raw).rolling(window=rolling_window,
min_periods=1).std().fillna(0).values
features = np.column_stack([data_raw, rolling_mean, rolling_std])

#Scaling
scaler = StandardScaler()
features_scaled = scaler.fit_transform(features)

#Sliding Windows
def create_windows(data, window_size):
    return np.array([data[i:i+window_size] for i in range(len(data) -
window_size + 1)])

X_all = create_windows(features_scaled, window_size)
y_all = labels[window_size-1:]

#Train using normal only
X_train = X_all[y_all == 0]

X_train_train, X_val = train_test_split(X_train, test_size=0.1,
random_state=seed)


input_shape = (window_size, X_all.shape[2])
initializer = initializers.HeNormal(seed=seed)

model = models.Sequential([
    layers.Input(shape=input_shape),
    layers.Conv1D(32, kernel_size=3, activation='relu',
padding='same', kernel_initializer=initializer),
    layers.Conv1D(16, kernel_size=3, activation='relu',
padding='same', kernel_initializer=initializer),
```

```
    layers.Conv1DTranspose(16, kernel_size=3, padding='same',
activation='relu', kernel_initializer=initializer),
    layers.Conv1DTranspose(32, kernel_size=3, padding='same',
activation='relu', kernel_initializer=initializer),
    layers.Conv1D(X_all.shape[2], kernel_size=1, activation='linear',
padding='same', kernel_initializer=initializer)
])

model.compile(optimizer='adam', loss='mse')
early_stop = EarlyStopping(monitor='val_loss', patience=5,
restore_best_weights=True, verbose=0)

model.fit(
    X_train_train, X_train_train,
    validation_data=(X_val, X_val),
    epochs=max_epochs,
    batch_size=128,
    shuffle=False,
    callbacks=[early_stop],
    verbose=0
)


X_pred = model.predict(X_all, verbose=0)
mse = np.mean(np.square(X_all - X_pred), axis=(1, 2))


#Threshold selection
best_f1 = 0
best_thresh = np.percentile(mse, 90)

for p in range(80, 100):
    thresh = np.percentile(mse, p)
    y_pred = (mse > thresh).astype(int)
    f1 = f1_score(y_all, y_pred)
    if f1 > best_f1:
        best_f1 = f1
        best_thresh = thresh

final_preds = (mse > best_thresh).astype(int)

end_time = time.time()
elapsed_time = end_time - start_time


result['prediction'] = np.concatenate([np.zeros(window_size-1),
final_preds]).astype(int)
result['reconstruction_error'] = np.concatenate([np.zeros(window_size-
1), mse])
```

```
print("\n Results:")
print(classification_report(labels, result['prediction']))

print(f"Precision: {precision_score(labels,
result['prediction']):.4f}")
print(f"Recall:    {recall_score(labels, result['prediction']):.4f}")
print(f"F1 Score:  {f1_score(labels, result['prediction']):.4f}")
print(f"ROC-AUC:   {roc_auc_score(labels,
result['reconstruction_error']):.4f}")
print(f"runtime: {elapsed_time / 60:.4f} minutes")
```

```
 Results:
              precision    recall  f1-score   support

           0       0.78      0.80      0.79     24629
           1       0.19      0.18      0.19      6569

    accuracy                           0.67     31198
   macro avg       0.49      0.49      0.49     31198
weighted avg       0.66      0.67      0.66     31198

Precision: 0.1921
Recall:    0.1824
F1 Score:  0.1871
ROC-AUC:   0.5319
runtime: 0.2714 minutes
```

AE NAIVE STREAMING

```
#parameters
window_size = 20
rolling_window = 10
batch_size = 5000
max_epochs = 100

data_raw = result['measurement'].values
labels = result['anomaly'].values
n_samples = len(data_raw)

all_preds = np.zeros(n_samples)
all_mse = np.zeros(n_samples)

initializer = initializers.HeNormal(seed=seed)

def create_windows(data, window_size):
    X = []
    for i in range(len(data) - window_size + 1):
```

```
        X.append(data[i:i+window_size])
    return np.array(X)

num_batches = n_samples // batch_size + (1 if n_samples % batch_size !
= 0 else 0)

start_time = time.time()

for b in range(num_batches):
    print(f"\n Processing Batch {b+1}/{num_batches}")

    start_idx = b * batch_size
    end_idx = min(start_idx + batch_size, n_samples)
    batch_raw = data_raw[start_idx:end_idx]
    batch_labels = labels[start_idx:end_idx]

    #Rolling mean & std
    rolling_mean = pd.Series(batch_raw).rolling(window=rolling_window,
min_periods=1).mean().values
    rolling_std = pd.Series(batch_raw).rolling(window=rolling_window,
min_periods=1).std().fillna(0).values

    #Features
    batch_features = np.column_stack([batch_raw, rolling_mean,
rolling_std])

    #Scaling
    scaler = StandardScaler()
    batch_features_scaled = scaler.fit_transform(batch_features)

    #Sliding windows
    if len(batch_features_scaled) < window_size:
        print("Skip batch: not enough data for sliding windows")
        continue

    X_batch_all = create_windows(batch_features_scaled, window_size)
    y_batch_true = batch_labels[window_size-1:]

    #Train using normal only
    X_batch_train = X_batch_all[y_batch_true == 0]

    if len(X_batch_train) < 10:
        print("Skip batch: too few normal samples for training")
        continue


    input_shape = (window_size, X_batch_all.shape[2])

    model = models.Sequential([
        layers.Input(shape=input_shape),
```

```
        layers.Conv1D(32, kernel_size=3, activation='relu',
padding='same', kernel_initializer=initializer),
        layers.Conv1D(16, kernel_size=3, activation='relu',
padding='same', kernel_initializer=initializer),
        layers.Conv1DTranspose(16, kernel_size=3, padding='same',
activation='relu', kernel_initializer=initializer),
        layers.Conv1DTranspose(32, kernel_size=3, padding='same',
activation='relu', kernel_initializer=initializer),
        layers.Conv1D(X_batch_all.shape[2], kernel_size=1,
activation='linear', padding='same', kernel_initializer=initializer)
    ])

    model.compile(optimizer='adam', loss='mse')

    X_train_train, X_val = train_test_split(X_batch_train,
test_size=0.1, random_state=seed)
    early_stop = EarlyStopping(monitor='val_loss', patience=5,
restore_best_weights=True, verbose=0)

    model.fit(
        X_train_train, X_train_train,
        validation_data=(X_val, X_val),
        epochs=max_epochs,
        batch_size=128,
        shuffle=False,
        callbacks=[early_stop],
        verbose=0
    )


    X_pred = model.predict(X_batch_all, verbose=0)
    mse = np.mean(np.square(X_batch_all - X_pred), axis=(1, 2))

    #Threshold search
    best_f1 = 0
    best_thresh = np.percentile(mse, 90)

    for p in range(80, 100):
        threshold = np.percentile(mse, p)
        y_pred = (mse > threshold).astype(int)
        f1 = f1_score(y_batch_true, y_pred)
        if f1 > best_f1:
            best_f1 = f1
            best_thresh = threshold

    y_batch_pred = (mse > best_thresh).astype(int)


    batch_pred_full = np.zeros(end_idx - start_idx)
    batch_mse_full = np.zeros(end_idx - start_idx)
```

```
    batch_pred_full[window_size-1:] = y_batch_pred
    batch_mse_full[window_size-1:] = mse

    all_preds[start_idx:end_idx] = batch_pred_full
    all_mse[start_idx:end_idx] = batch_mse_full


end_time = time.time()
elapsed_time = end_time - start_time


result['prediction'] = all_preds.astype(int)
result['reconstruction_error'] = all_mse

print("\nResults:")
print(classification_report(labels, all_preds.astype(int)))

print(f"Precision: {precision_score(labels,
all_preds.astype(int)):.4f}")
print(f"Recall:    {recall_score(labels, all_preds.astype(int)):.4f}")
print(f"F1 Score:  {f1_score(labels, all_preds.astype(int)):.4f}")
print(f"ROC-AUC:   {roc_auc_score(labels, all_mse):.4f}")
print(f"runtime: {elapsed_time / 60:.4f} minutes")
```

```
 Processing Batch 1/7

 Processing Batch 2/7

 Processing Batch 3/7

 Processing Batch 4/7

 Processing Batch 5/7

 Processing Batch 6/7

 Processing Batch 7/7

Results:
              precision    recall  f1-score   support

           0       0.83      0.89      0.86     24629
           1       0.43      0.30      0.35      6569

    accuracy                           0.77     31198
   macro avg       0.63      0.60      0.61     31198
weighted avg       0.74      0.77      0.75     31198
```

```
Precision: 0.4252
Recall:    0.3043
F1 Score:  0.3547
ROC-AUC:   0.5880
runtime: 1.0334 minutes
```

AE STREAMING VARIANT 2

```
#Parametes
window_size = 20
rolling_window = 10
batch_size = 5000
max_epochs = 100

data_raw = result['measurement'].values
labels = result['anomaly'].values
n_samples = len(data_raw)

all_preds = np.zeros(n_samples)
all_mse = np.zeros(n_samples)

initializer = initializers.HeNormal(seed=seed)

def create_windows(data, window_size):
    X = []
    for i in range(len(data) - window_size + 1):
        X.append(data[i:i+window_size])
    return np.array(X)

num_batches = n_samples // batch_size + (1 if n_samples % batch_size !
= 0 else 0)

model = None


start_time = time.time()

for b in range(num_batches):
    print(f"\nProcessing Batch {b+1}/{num_batches}")

    start_idx = b * batch_size
    end_idx = min(start_idx + batch_size, n_samples)
    batch_raw = data_raw[start_idx:end_idx]
    batch_labels = labels[start_idx:end_idx]

    #Rolling mean & std
    rolling_mean = pd.Series(batch_raw).rolling(window=rolling_window,
min_periods=1).mean().values
```

```
    rolling_std = pd.Series(batch_raw).rolling(window=rolling_window,
min_periods=1).std().fillna(0).values

    #Features
    batch_features = np.column_stack([batch_raw, rolling_mean,
rolling_std])

    #Scaling
    scaler = StandardScaler()
    batch_features_scaled = scaler.fit_transform(batch_features)

    #Sliding windows
    if len(batch_features_scaled) < window_size:
        print("Skip batch: not enough data for sliding windows")
        continue

    X_batch_all = create_windows(batch_features_scaled, window_size)
    y_batch_true = batch_labels[window_size-1:]

    #Train using normal only
    X_batch_train = X_batch_all[y_batch_true == 0]

    if len(X_batch_train) < 10:
        print("Skip batch: too few normal samples for training")
        continue

    input_shape = (window_size, X_batch_all.shape[2])

    if model is None:
        model = models.Sequential([
            layers.Input(shape=input_shape),
            layers.Conv1D(32, kernel_size=3, activation='relu',
padding='same', kernel_initializer=initializer),
            layers.Conv1D(16, kernel_size=3, activation='relu',
padding='same', kernel_initializer=initializer),
            layers.Conv1DTranspose(16, kernel_size=3, padding='same',
activation='relu', kernel_initializer=initializer),
            layers.Conv1DTranspose(32, kernel_size=3, padding='same',
activation='relu', kernel_initializer=initializer),
            layers.Conv1D(X_batch_all.shape[2], kernel_size=1,
activation='linear', padding='same', kernel_initializer=initializer)
        ])
        model.compile(optimizer='adam', loss='mse')


    X_train_train, X_val = train_test_split(X_batch_train,
test_size=0.1, random_state=seed)
    early_stop = EarlyStopping(monitor='val_loss', patience=5,
restore_best_weights=True, verbose=0)
```

```
    model.fit(
        X_train_train, X_train_train,
        validation_data=(X_val, X_val),
        epochs=max_epochs,
        batch_size=128,
        shuffle=False,
        callbacks=[early_stop],
        verbose=0
    )


    X_pred = model.predict(X_batch_all, verbose=0)
    mse = np.mean(np.square(X_batch_all - X_pred), axis=(1, 2))

    #Threshold search
    best_f1 = 0
    best_thresh = np.percentile(mse, 90)

    for p in range(80, 100):
        threshold = np.percentile(mse, p)
        y_pred = (mse > threshold).astype(int)
        f1 = f1_score(y_batch_true, y_pred)
        if f1 > best_f1:
            best_f1 = f1
            best_thresh = threshold

    y_batch_pred = (mse > best_thresh).astype(int)


    batch_pred_full = np.zeros(end_idx - start_idx)
    batch_mse_full = np.zeros(end_idx - start_idx)

    batch_pred_full[window_size-1:] = y_batch_pred
    batch_mse_full[window_size-1:] = mse

    all_preds[start_idx:end_idx] = batch_pred_full
    all_mse[start_idx:end_idx] = batch_mse_full



end_time = time.time()
elapsed_time = end_time - start_time


result['prediction'] = all_preds.astype(int)
result['reconstruction_error'] = all_mse

print("\nResults:")
print(classification_report(labels, all_preds.astype(int)))

print(f"Precision: {precision_score(labels,
```

```
all_preds.astype(int)):.4f}")
print(f"Recall:    {recall_score(labels, all_preds.astype(int)):.4f}")
print(f"F1 Score:  {f1_score(labels, all_preds.astype(int)):.4f}")
print(f"ROC-AUC:   {roc_auc_score(labels, all_mse):.4f}")
print(f"runtime: {elapsed_time / 60:.4f} minutes")
```

```
Processing Batch 1/7

Processing Batch 2/7

Processing Batch 3/7

Processing Batch 4/7

Processing Batch 5/7

Processing Batch 6/7

Processing Batch 7/7

Results:
              precision    recall  f1-score   support

           0       0.83      0.90      0.86     24629
           1       0.45      0.32      0.37      6569

    accuracy                           0.77     31198
   macro avg       0.64      0.61      0.62     31198
weighted avg       0.75      0.77      0.76     31198

Precision: 0.4479
Recall:    0.3171
F1 Score:  0.3713
ROC-AUC:   0.5002
runtime: 0.4498 minutes
```